

Mock Design: Rate Limiter

Phase 4 · Interview-Ready · Estimated time: 1.5 hours

1. Concept From First Principles: Applying the Framework

Phase 2, Chapter 10 already taught the core rate-limiting algorithms (token bucket, sliding window, fixed window) in isolation. This chapter's job is different: it's about running Chapter 1's full interview framework around that existing knowledge, live, under time pressure — because "design a rate limiter" is asked constantly precisely because it's a compact problem that still has real depth in its distributed-systems deep dive.

Step 1 — Clarify requirements

- Functional: limit each client (by user ID, API key, or IP) to N requests per time window; return a clear rejection (HTTP 429) when exceeded.
- Non-functional: the limiter itself must not become the bottleneck — it should add minimal latency, and it must work correctly across multiple application servers, not just one.
- Clarify scope explicitly: is this a rate limiter for one API endpoint, or a shared platform-wide service used by many internal teams (a genuinely different, harder problem)?

Step 2 — Estimate scale

Assume a platform-wide rate limiter fronting 500,000 requests/second across all APIs, each request requiring a limiter check. This check must complete in low single-digit milliseconds, since it sits on the critical path of every single request — a rate limiter that adds 50ms of latency to every request defeats its own purpose.

Step 3 — API / interface

Typically implemented as middleware rather than a separate HTTP call for performance, but if exposed as a service: `check(clientId, resource) -> {allowed: bool, retryAfter: seconds}`. State clearly that this check must happen synchronously, before the actual request is processed.

Step 4 — High-level design

The critical decision, and the one interviewers probe hardest: where does rate-limit state live?

In-memory per server is fastest but wrong at scale — a client hitting a load-balanced fleet of 10 servers effectively gets a 10x higher limit than intended, since each server tracks its own count independently. The correct answer for a multi-server deployment is a **centralized, shared store** — Redis, chosen specifically for its atomic increment operations and sub-millisecond latency (Phase 1, Ch 8) — so every server checks and updates the same shared counter per client. Request flow: Client → Load Balancer → API server → (before processing) check-and-increment against Redis → allow or reject.

Step 5 — Deep dive: algorithm choice and distributed correctness

Revisit Phase 2 Ch 10's algorithms with the distributed lens this interview format demands. **Fixed window counter** is simplest (increment a counter, reset every N seconds) but has a real edge burst flaw: a client can send double the intended limit by timing requests around a window boundary (e.g., the last second of one window and the first second of the next). **Sliding window log** (store a timestamp per request, count requests within the trailing window) is accurate but memory-heavy at high request volume. **Token bucket** is the most commonly chosen production algorithm because it naturally allows brief bursts up to the bucket size while still enforcing a steady long-term rate, and its state (just a token count and last-refill timestamp) is cheap to store and atomically update in Redis using a single Lua script — critical for avoiding race conditions when many application servers check the same key concurrently, since a naive read-then-write from multiple servers can both pass a check that should have failed. Naming this exact race condition, and that a Redis Lua script (or Redis's atomic INCR/EXPIRE commands) resolves it, is a strong, specific deep-dive answer.

Step 6 — Trade-offs and wrap-up

Volunteer: the shared Redis store is now a single point of failure for every rate-limited request platform-wide — discuss Redis clustering/replication (Phase 1, Ch 3) as the mitigation, and note the explicit fail-open vs. fail-closed decision if Redis itself becomes unavailable (most production systems choose to fail open — allow requests through — rather than taking down the entire platform because the rate limiter's dependency failed, since availability of the core product usually outweighs strict enforcement of a secondary protective mechanism).

2. Real-World Example / Case Study

Razorpay's and PhonePe's public APIs enforce per-merchant rate limits to protect core payment infrastructure from both accidental retry storms and deliberate abuse — a direct, production instance of exactly this design. Interviewers at fintech-adjacent Indian companies frequently choose this problem specifically because correct distributed rate limiting is a genuine production concern they've solved themselves.

3. Diagram

```
[Client] --request--> [Load Balancer]
                        |
                    (fanned out across many API servers)
                        |
                    [ API Server N ]
                        |
                    check-and-increment (atomic)
                        v
                    [ Shared Redis Store ]
                    key: clientId, algorithm state
                    (token bucket: tokens, last_refill)
                        |
                    allowed? --> process request
                    denied? --> HTTP 429 + Retry-After
```

Fail-open policy: if Redis is unreachable, allow the request through rather than blocking all traffic.

4. Trade-offs Table

Decision	Alternative	Why this choice
Centralized Redis-backed state	Per-server in-memory counters	Per-server counters undercount the effective limit across a fleet
Token bucket algorithm	Fixed window counter	Avoids the boundary-burst flaw while still allowing intended short bursts
Atomic Lua script / INCR+EXPIRE	Naive read-then-write in app code	Prevents race conditions from concurrent servers checking the same key
Fail-open on Redis outage	Fail-closed (block all traffic)	Availability of core product outweighs strict enforcement of a protective layer

5. Common Interview Questions

Q1: "Why can't each application server just track its own rate-limit counters?"

Model approach: Explain that with a load-balanced fleet, a client's requests are spread across many servers, so per-server counters let a client effectively multiply their limit by the number of servers — a centralized shared store is required for a correct platform-wide limit.

Q2: "How do you prevent a race condition when two servers check the same client's limit at the same instant?"

Model approach: Name the specific mechanism — an atomic Redis operation (INCR+EXPIRE, or a Lua script bundling check-and-decrement into one atomic step) — rather than a read-then-write pattern in application code, which has a race window between the read and the write.

Q3: "What happens to your whole platform if the Redis store rate limiting depends on goes down?"

Model approach: State the fail-open vs. fail-closed decision explicitly and justify it — fail-open (allow traffic through) is the common production choice, since the rate limiter is a protective secondary system and shouldn't itself become a cause of a full outage.

6. Practice Exercises

- 1 Run the full six-step framework for "design a rate limiter for a public API," out loud, with a timer.
- 2 Write the pseudocode for a Redis Lua script implementing token-bucket check-and-consume atomically.
- 3 Explain, from memory, why sliding window log is more accurate but more expensive than fixed window counter, referencing Phase 2 Chapter 10.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"System Design Interview: Rate Limiting Algorithms That Get You Hired"	Directly covers the algorithm trade-offs this chapter's deep dive relies on.
English	"Master Rate Limiting - System Design"	Reinforces the distributed-correctness angle (shared state, atomic operations).
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Bilingual coverage including rate limiting fundamentals.
Hindi	"Complete System Design Roadmap 2026 HLD & LLD" — CodeHelp	Hindi roadmap covering rate limiting as part of the broader curriculum.

8. Key Takeaways

- This chapter applies Chapter 1's framework to Phase 2 Ch 10's algorithms — interview success here is about distributed correctness, not just naming an algorithm.
- Per-server in-memory counters undercount the effective limit across a load-balanced fleet; centralized shared state is required.
- Token bucket is the most commonly chosen production algorithm: cheap state, natural burst tolerance, steady long-term rate.
- Fixed window counters have a real boundary-burst flaw; sliding window log is accurate but memory-heavy.
- Atomic operations (Lua scripts, INCR+EXPIRE) are required to avoid race conditions between concurrent servers checking shared state.
- The fail-open vs. fail-closed decision on limiter-dependency outage is a genuine, discussable trade-off, not an afterthought.
- Volunteering the shared store's single-point-of-failure risk and its mitigation (clustering/replication) is a strong Step 6 answer.